NM-AIST / SSE+ 0003

Introduction to Programming and Python

Unit 1 • Foundations, problem-solving, professional habits and AI-assisted learning

PYTHON AND R | STUDENT LEARNING MATERIAL

Learning outcomes

- Explain Python, its uses, strengths, limitations and paradigms.
- Create, save and run a script using variables, comments and print.
- Design an algorithm, trace a calculation, test and document the result.
- Use version history and AI assistance responsibly.

Programming foundations

- A program expresses instructions that a computer can execute. Programming includes understanding a problem, designing its solution and checking whether the result is correct.

Unit 1 explores these ideas through everyday calculations. Python provides the notation for the first working examples.

Programs in everyday life

- A calculator computes results, a banking application records transactions, and a registration system organises student information. Each uses instructions designed for a particular task.

A small transport-cost calculator follows the same basic principle: supplied information is processed to produce a useful result.

What is Python?

Python is a high-level, general-purpose programming language created by Guido van Rossum and first released in 1991. Its readable notation allows people to express instructions without writing machine instructions.

Python 3 is used in this course. Python is free and open source, and runs on Windows, macOS and Linux. A Python script usually has the .py extension.

CODE / EXAMPLE

```
print("Hello, Python!")
```

EXPECTED OUTPUT

```
Hello, Python!
```

Why learn Python?

- Python is approachable for beginners, supports rapid development and has a large ecosystem of libraries and learning resources.

- Python is used in automation, web back ends, scientific computing, data analysis and AI. For example, software can analyse crop images, process banking data or predict demand. NumPy, pandas and scikit-learn support later scientific and machine-learning work; none is required in Unit 1.

The simple calculations in this unit are the building blocks of larger solutions.

Python strengths and limitations

- Strengths include readable code, portability, rapid experimentation and extensive libraries. Python is useful for prototypes and production systems.
- Limitations include slower pure-Python loops than compiled C/C++ for some compute-heavy tasks, runtime type mistakes, and managing dependencies. Optimised native libraries can improve performance. Python is not usually the first choice for hard real-time embedded systems or native mobile apps.

Choose tools according to the problem, correctness, performance and deployment needs.

Programming paradigms

- A programming paradigm is an approach to organising a solution. Imperative programming specifies actions that change state. Procedural programming is an imperative approach that groups actions into procedures or functions.

- Object-oriented programming organises related data and behaviour into objects. Functional programming emphasises functions and transformations of data, often avoiding unnecessary state changes. A program may combine these approaches.

Procedural programming

A procedural solution describes steps in order and groups repeated operations into reusable functions. It is useful for calculations and tasks with a clear sequence.

The course starts with sequential imperative scripts and introduces functions in Unit 5. A short calculation does not require classes.

CODE / EXAMPLE

```
fare = 1500
trips = 6
total = fare * trips
print(total)
```

EXPECTED OUTPUT

```
9000
```

Object-oriented programming (OOP)

- An object combines data and behaviour. A class is a blueprint for creating objects. A Student object may hold a name and registration number and provide a method to update contact details.

- OOP helps organise related behaviour in larger systems, but classes may make a tiny calculation unnecessarily complex. Writing classes is not required in this unit.

Example: Student, Course and Registration objects in a university system.

Functional programming and Python

- Functional programming emphasises transforming inputs into outputs using functions. A pure function returns the same result for the same inputs and has no side effects.
- Python is multi-paradigm: it supports procedural, object-oriented and functional-style programming. It is not restricted to any one paradigm. This course begins with sequential scripts, then reusable functions.

Choose the approach that makes the solution clear, testable and maintainable.

One problem, three approaches

A transport calculator can be organised in different ways. Python supports all three approaches.

Procedural

Store the fare and trips, calculate the total, then display it. Reusable procedures can group the steps.

Object-oriented

A Trip object holds its fare and number of trips. A method calculates its total cost.

Functional

A function receives fare and trips and returns their product, without changing the inputs.

For a first four-line program, the procedural approach is enough. Classes and advanced functional tools can wait.

How Python runs code

- A language defines notation and rules (syntax). An interpreter processes and executes instructions. Common CPython implementations compile source to bytecode and execute it in the Python runtime; internal details are not required here.
- An editor writes and edits source code. An IDE combines editing, running and debugging. A terminal accepts text commands. A console displays output or accepts interactive commands.

Source code → Python runtime → output. Writing and executing are different actions.

Getting Python ready

Install Python 3 from python.org/downloads using the instructions for your operating system. Windows installations may use py; others use python or python3.

Open a terminal and check one of the commands below. A Python 3 version confirms that the command finds Python. If it fails, check installation and PATH or ask for help. Do not type terminal commands at the Python shell prompt.

TERMINAL COMMANDS

```
python --version
# Alternatives:
# py --version
# python3 --version
```

EXPECTED OUTPUT

```
Python 3.x.x (example)
```

Your first interactive command

IDLE is a simple editor and shell included with many Python installations. Open IDLE or another installed Python environment. At the shell prompt >>>, type the command below and press Enter.

The shell executes commands immediately. Do not copy the >>> prompt into a script. Google Colab is an alternative online environment; export a .py file when the assignment requires a script.

CODE / EXAMPLE

```
print("Hello, Python!")
```

EXPECTED OUTPUT

```
Hello, Python!
```

Create, save and run a script

In IDLE choose File → New File, enter the code, and save it as hello.py inside SSE_Python/Unit1. Choose Run → Run Module (F5). The output appears in the shell.

Alternatively, in a terminal navigate to the containing folder and run python hello.py (or py hello.py / python3 hello.py). Save before running. Check the extension is .py, not .py.txt.

CODE / EXAMPLE

```
print("Hello, Python!")
print(10 + 5)
```

EXPECTED OUTPUT

```
Hello, Python!
15
```

Algorithms

- An algorithm is a clear, ordered sequence of steps for solving a problem. It can be expressed in ordinary language before code is written.

Notebook example: identify the price and quantity, multiply them, then report the total. Three notebooks at TZS 2,000 cost TZS 6,000.

Pseudocode and flowcharts

An algorithm can be represented using pseudocode (structured human-readable steps) or a flowchart (a diagram). Pseudocode is not executable Python.

Flowcharts use ovals for start/end, parallelograms for input/output, rectangles for processing and diamonds for decisions. Arrows show the order of execution.

PSEUDOCODE (NOT PYTHON)

```
START
  SET price TO 2000
  SET quantity TO 4
  SET total TO price × quantity
  DISPLAY total
END
```

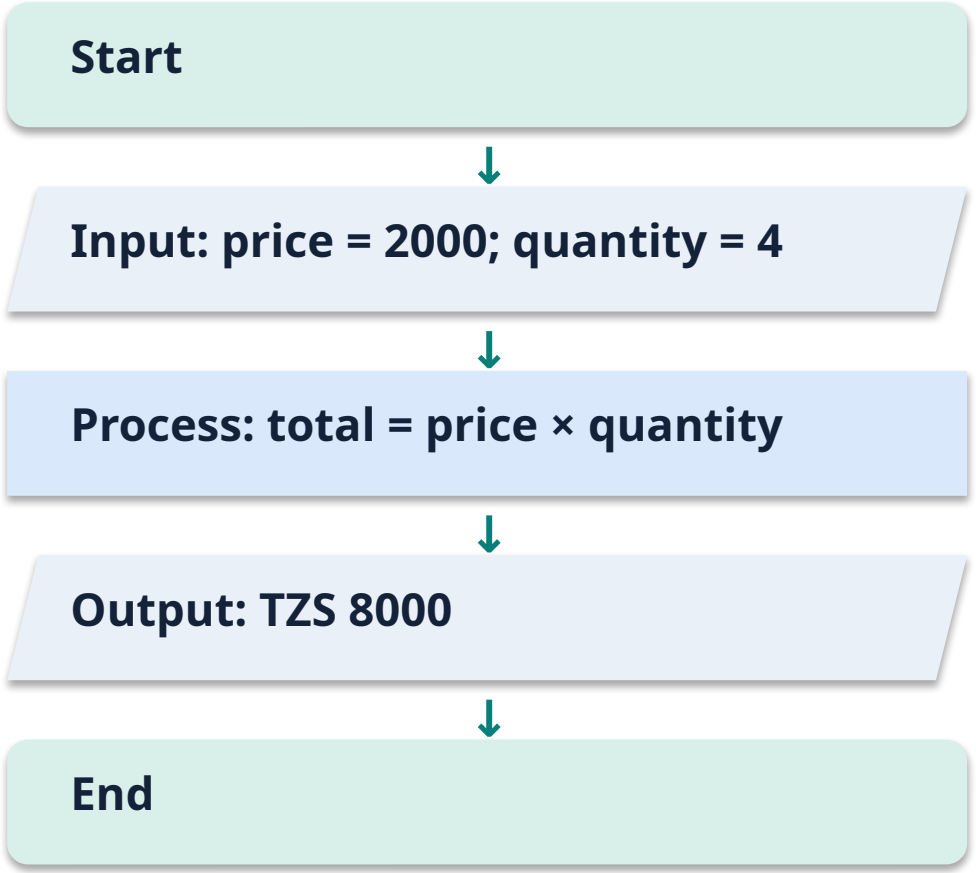
Start → input price and quantity → multiply → display → End.

Flowchart: notebook cost

A flowchart represents an algorithm visually. The arrows show the sequence; each shape has a purpose.

The price and quantity are fixed here. The processing step multiplies them, and the output reports the cost in TZS.

The equivalent Python script uses assignment, multiplication and print().



Inputs processing and outputs

- Inputs supply information. Processing performs operations on that information. Outputs communicate the results.

Transport example
Input: fare 1500 and trips 6
Processing: 1500 × 6
Output: TZS 9000

Assumptions in a solution

- The transport calculation assumes six one-way trips, the same fare for each trip and no extra charges. These conditions define when the calculation is suitable.

Different evening and morning fares would require a revised model. Correct arithmetic cannot fix an incorrect assumption.

Sequence

- The fare and number of trips must exist before the program can multiply them. The total must be calculated before it can be displayed.

1. Set the fare
2. Set the number of trips
3. Multiply fare by trips
4. Display the total and units

First calculations

Python evaluates the expression inside print() before displaying it. The symbols +, - and * mean addition, subtraction and multiplication.

Use whole numbers without thousands separators. Unit 2 develops operators, data types and typed input more systematically.

CODE / EXAMPLE

```
print(10 + 5)
print(10000 - 3000)
print(1500 * 6)
```

EXPECTED OUTPUT

```
15
7000
9000
```

The first Python program

Python executes this example from top to bottom. The first two statements establish the inputs. The third calculates the total. The final statement displays it.

CODE / EXAMPLE

```
fare = 1500
trips = 6
total = fare * trips
print("Cost in TZS:", total)
```

EXPECTED OUTPUT

```
Cost in TZS: 9000
```

Variables and assignment

- A variable is a name associated with a value. The statement fare = 1500 assigns 1500 to the name fare. Meaningful names explain what the values represent.

In total = fare * trips, Python evaluates the expression on the right and assigns its result to total. The equals sign performs assignment.

Changing values

- If trips becomes 8 and the calculation runs again, the total becomes 12000. The same calculation can answer a new case.

A previously calculated total does not update automatically when trips changes. The multiplication statement must execute again.

Text and numbers

A string is text enclosed in quotation marks. A label such as "Total cost:" helps explain a result. An integer such as 1500 is a whole-number value.

CODE / EXAMPLE

```
total = 9000
print("total")  # displays total
print(total)    # displays 9000
```

EXPECTED OUTPUT

```
total
9000
```

Clear output

The print() function displays values. Parentheses contain its supplied values, and a comma separates the label from the value in this example.

CODE / EXAMPLE

```
total = 9000
print("Cost in TZS:", total)
```

EXPECTED OUTPUT

```
Cost in TZS: 9000
```

Good practice: names and comments

Use descriptive names such as fare_per_trip. Python names are case-sensitive, cannot contain spaces or begin with a digit, and cannot be reserved words. Use snake_case for variables and consistent spacing.

A comment begins with # outside a string and continues to the end of the line. Comments explain assumptions or decisions. Python uses indentation for blocks; four spaces is the usual convention.

CODE / EXAMPLE

```
# All trips have the same fare.
fare_per_trip = 1500
number_of_trips = 6
total_cost = fare_per_trip *
number_of_trips
print(total_cost)
```

EXPECTED OUTPUT

```
9000
```

Good practice: saving and organising

- Keep a course folder with subfolders for each unit. Save after meaningful changes and before running. Use descriptive filenames, and keep working files separate from submission copies.
- Back up important work to an approved cloud location or another drive. Never put passwords or confidential records in code or public repositories.

A well-organised project can be reopened, run and checked by another person.

Good practice: version history

- A version is a recorded state of a project. Version history lets you compare changes, recover an earlier working state and understand why a change was made.
- Keep a working copy and named milestones such as transport_v1.py and transport_v2.py with a short change log. Git records commits; GitHub hosts repositories and supports collaboration. Git commands are not required in Unit 1.

Backups protect against loss; version control records how work evolves.

Good practice: documentation

- Documentation explains purpose, inputs, assumptions, how to run the program and expected results. A short README or notes file allows someone else to reproduce your work.
- For the transport calculator, record Python 3, the filename, fixed-fare assumption, example values, expected output and tests. Comments explain local details; documentation explains the whole program.

Update documentation when the code changes.

Tracing execution

- Tracing follows each instruction and records the values produced. It explains how the final result was reached.

After line 1: fare is 1500
After line 2: trips is 6
After line 3: total is 9000
After line 4: the result is displayed

Test cases

- A test case combines input values with an independently calculated expected result. Comparing that expectation with the actual output checks the program for that case.

1500 × 6 should produce 9000
1500 × 8 should produce 12000
1500 × 0 should produce 0

Passing a few tests does not prove every case will work.

Errors and debugging

- A syntax error breaks writing rules. A NameError can refer to an unknown name. A logic error can produce an incorrect answer even when the program runs.

Missing quotation mark: syntax error
Fare instead of defined fare: NameError
fare + trips instead of multiplication: logic error

AI as a learning assistant

- AI tools can explain concepts, suggest examples, offer hints, interpret errors and review code. They may also produce incorrect or insecure code, invent features or miss assumptions.
- Understanding comes first: attempt the algorithm, predict the result, run the code yourself and explain every line. Do not submit code you cannot explain or modify.

Use AI to improve your reasoning, not replace it.

A productive AI-assisted workflow

- 1. State the problem and your current understanding. 2. Attempt an algorithm. 3. Ask for a hint or explanation of a specific obstacle. 4. Verify the suggestion, run tests and revise. 5. Document assistance.

- Example prompt: “I am learning variables. My transport calculator uses fare + trips. I expected 9000 but got 1506. Explain why without giving the complete assignment solution.”

A confident AI answer is not evidence of correctness. Check documentation and test independently.

AI, privacy and academic integrity

- Do not paste passwords, confidential records or identifying information into AI tools. Use fictional examples and follow course and institutional rules on permitted assistance.
- Declare the tool and how it helped when requested. Example: “Used AI to explain a NameError; corrected the variable name and independently tested three cases.” If you cannot explain a suggestion, practise before using it.

Practice: combined cost

Two books cost TZS 1,500 each and three pens cost TZS 500 each. Calculate two subtotals, then add them. Predict the result before running.

Change the quantities and calculate the new expected answer by hand. Save the script with a meaningful filename.

CODE / EXAMPLE

```
book_cost = 2 * 1500
pen_cost = 3 * 500
total = book_cost + pen_cost
print("Total in TZS:", total)
```

EXPECTED OUTPUT

```
Total in TZS: 4500
```

Practice: remaining allowance

A student has TZS 10,000 and spends TZS 3,000 on transport and TZS 2,500 on food. Subtract both costs from the allowance.

Explain each variable and modify one input. A negative remainder would indicate overspending; conditions and validation come later.

CODE / EXAMPLE

```
allowance = 10000
transport = 3000
food = 2500
remaining = allowance - transport - food
print("Remaining:", remaining)
```

EXPECTED OUTPUT

```
Remaining: 4500
```

Guided practice

- Shopping: four notebooks cost TZS 2,000 each. The total is 8000. Seven notebooks at the same price cost 14000.

Study time: 30 minutes on five days totals 150 minutes. Changing the values to 45 minutes on four days produces 180 minutes.

Independent activity U1

- An exercise-book calculator uses a price and quantity to calculate total cost. The submission includes a problem description, an algorithm, runnable code and test evidence.

Required cases
1500 and 3: 4500
2500 and 4: 10000
2000 and 0: 0

The activity page contains the full brief and marking criteria.

Readiness for the next unit

- A clear explanation connects the problem, its algorithm and the code. Test evidence connects expected results with actual outputs.

Unit 1 readiness means being able to explain a simple calculation, modify its values, save the program and identify a result that is incorrect.